Lost in Pieces: An Image Jigsaw Reconstruction Challenge

Τεχνική Αναφορά — Ανάλυση Εικόνας

Δημήτρης Συμεωνίδης (AM: Π22165)

February, 2026

Contents

1 Εισαγωγή **3**

2 Εγκατάσταση και Εκτέλεση **3**
2.1 Απαιτήσεις Συστήματος . . . 3
2.2 Εγκατάσταση Dependencies . . . 3
2.3 Δομή Project . . . 3
2.4 Εντολές Εκτέλεσης . . . 4
2.5 Δομή Αρχείων Εξόδου . . . 4

3 Προετοιμασία Δεδομένων — Stage 1 **5**
3.1 Γιατί Τετράγωνα Tiles . . . 5
3.2 Δυναμικό Grid . . . 5
3.3 Ανακάτεμα και Περιστροφή . . . 5

4 Εξαγωγή Χαρακτηριστικών — Stage 2 **6**
4.1 Ιστογράμματα Χρώματος (Φ_{col}) . . . 6
4.2 Υφή — Gabor + Sobel (Φ_{tex}) . . . 6
4.3 Edge Pixels . . . 6
4.4 Harris + SIFT (Φ_{loc}) . . . 7
4.5 Deep CNN (Φ_{deep}) . . . 7
4.6 Πίνακας Descriptors . . . 7
4.7 Ανάλυση avg_tile_histogram . . . 7
4.8 Log Output — Stage 2 . . . 7

5 Compatibility Score **7**

6 Αλγόριθμος Επίλυσης — Stage 3 **8**
6.1 Greedy Construction + Exhaustive Start . . . 8
6.2 Local Search . . . 8
6.3 Compatibility Matrix — Heatmap Ανάλυση . . . 8
6.4 Rotation-Aware Evaluation . . . 9
6.5 evaluation.txt — Μορφή Αρχείου Εξόδου . . . 9

7 Πειραματικά Αποτελέσματα **10**
7.1 Batch Run — 7 Εικόνες . . . 10
7.2 Αναλυτικά ανά Εικόνα . . . 10
7.3 Οπτική Επιβεβαίωση — Solved Puzzles . . . 11
7.4 Πλήρες Log Output (I2) . . . 11

8 Ανάλυση Αποτυχίας — egg **12**
8.1 Η Εικόνα 12
8.2 Γιατί Αποτυγχάνει Μαθηματικά 12

9 Ablation Study **13**
9.1 Σχεδιασμός 13
9.2 Io — Ευρεία Αξιολόγηση (Επιτυχημένη Εικόνα) 13
9.3 egg — Ευρεία Αξιολόγηση (Αποτυχημένη Εικόνα) 14
9.4 ablation_results.csv 16

10 Machine Learning — Γιατί Δεν Υλοποιήθηκε **16**
10.1 Γιατί Δεν Χρειάζεται 16
10.2 Γιατί Δεν Θα Βοηθούσε στο egg 16
10.3 Τι Θα Μπορούσε να Βοηθήσει 17

11 Interactive GUI **17**

12 Συμπεράσματα **18**

1 Εισαγωγή

Το πρόβλημα *Jigsaw Puzzle Reconstruction* ζητά την αυτόματη ανακατασκευή μιας εικόνας από τα τεμαχισμένα, ανακατεμένα και τυχαία περιστραμμένα κομμάτια της — χωρίς γνώση της αρχικής διάταξης. Πρόκειται για ένα NP-hard πρόβλημα βελτιστοποίησης: για N τετράγωνα tiles με 4 πιθανές περιστροφές, ο χώρος αναζήτησης είναι $N! \times 4^N$ — δηλαδή $\sim 2 \times 10^{20}$ για μόλις 12 tiles.

Η προσέγγισή μας συνδυάζει κλασικά (ιστογράμματα χρώματος, Gabor/Sobel, edge pixels, Harris+SIFT) και deep χαρακτηριστικά (ResNet-18 features), υπολογίζει compatibility scores μεταξύ γειτονικών ακμών, και λύνει το puzzle με greedy construction + exhaustive start + local search.

Ο αλγόριθμος πέτυχε 100% placement, rotation και neighbor accuracy σε 6 από 7 test εικόνες. Η 7η (`egg.jpg`) αποτυγχάνει με 16.7% placement — και η ανάλυση αυτής της αποτυχίας αποδεικνύει ότι πρόκειται για underdetermined πρόβλημα (μηδενική πληροφορία στις ραφές), όχι αδυναμία του αλγορίθμου.

2 Εγκατάσταση και Εκτέλεση

2.1 Απαιτήσεις Συστήματος

- **Python** ≥ 3.8
- **pip** (ή conda) για εγκατάσταση πακέτων
- **GPU:** προαιρετικό — ο ResNet-18 τρέχει και σε CPU ($\sim$10s ανά εικόνα)

2.2 Εγκατάσταση Dependencies

```
pip install numpy opencv-python matplotlib
pip install torch torchvision            # CPU version
# ή: pip install torch torchvision --index-url https://download.pytorch.org/whl/
    cu121  # GPU
```

Πλήρης λίστα:

Package	Version	Purpose
numpy	$\geq$1.21	arrays, math
opencv-python	$\geq$4.5	image I/O, filters, color conversion
torch	$\geq$1.12	ResNet-18 deep features
torchvision	$\geq$0.13	pretrained models, transforms
matplotlib	$\geq$3.5	plots, histograms, heatmaps
tkinter	built-in	GUI (included in Python)

2.3 Δομή Project

```
puzzle_project/
  input_images/          # test images (jpg/png)
  results/               # output
  puzzle_utils.py        # load, crop, shuffle, rotate
  features.py            # 5 descriptor families
  solver.py              # compatibility, greedy, local search
  ablation.py            # feature configuration comparison
  main.py                # CLI pipeline
  gui.py                 # interactive Tkinter GUI
```

Ο κώδικας χωρίζεται σε modules — ο solver δεν έχει ποτέ πρόσβαση στη σωστή λύση.

2.4 Εντολές Εκτέλεσης

```
# Single image
python main.py --image I2.jpg --seed 42

# All images (batch)
python main.py --all --seed 42

# Batch + ablation study
python main.py --all --seed 42 --ablation

# Ablation only (requires previous run for features)
python ablation.py --image egg.jpg --seed 42

# Clean previous results and re-run
python main.py --all --seed 42 --clean-results

# Interactive GUI
python gui.py
```

Listing 1: CLI commands

Arguments:

- `--image FILE`: single image from `input_images/`
- `--all`: batch mode — process all images
- `--seed N`: random seed for reproducibility (default: 42)
- `--ablation`: run ablation study after solving
- `--clean-results`: delete `results/` before running

2.5 Δομή Αρχείων Εξόδου

Κάθε εκτέλεση δημιουργεί `results/[imagefile]/` με τα εξής:

```
results/[imagefile]/
  stage1.pkl
  stage2.pkl
  Stage_1_Shuffling/
    original_cropped.jpg          # cropped image
    shuffled_puzzle.jpg           # shuffled image
  Stage_2_Features/
    avg_tile_histogram.png        # average tile histogram
  Stage_3_Solving/
    solved_puzzle.jpg             # solved puzzle
    compatibility_matrix.png      # compatibility matrix
    evaluation.txt                # metrics (Pos/Rot/Neigh %)
  Stage_4_Ablation/               # (only if ablation is run)
    ablation_results.csv          # results in CSV
    ablation_comparison.png       # bar chart comparison
    classical/solved_classical.jpg
    deep_only/solved_deep_only.jpg
    no_local/solved_no_local.jpg
    combined/solved_combined.jpg
```

Τα features αποθηκεύονται σε pickle (`stage2.pkl`) ώστε η ablation study να τα επαναχρησιμοποιεί χωρίς νέο υπολογισμό.

3 Προετοιμασία Δεδομένων — Stage 1

3.1 Γιατί Τετράγωνα Tiles

Η εκφώνηση ζητά περιστροφές $\theta \in \{0°, 90°, 180°, 270°\}$. Ένα ορθογώνιο 80×120 μετά από 90° γίνεται 120×80 — δεν χωράει πίσω εάν δεν παραμορφωθεί. Λύση: center crop ώστε $h = w$.

3.2 Δυναμικό Grid

Ο αριθμός γραμμών/στηλών υπολογίζεται αυτόματα από το aspect ratio, με ελάχιστο 3×3:

```python
def compute_dynamic_grid(h, w):
    rho = h / w
    if   rho >= 1.35: rows, cols = 5, 3   # portrait
    elif rho >= 1.15: rows, cols = 4, 3
    elif rho <= 0.74: rows, cols = 3, 5   # landscape
    elif rho <= 0.87: rows, cols = 3, 4
    else:             rows, cols = 3, 3
    # Clamp -- tiles >= 96px, grid >= 3x3
    while rows > 3 and h // rows < 96: rows -= 1
    while cols > 3 and w // cols < 96: cols -= 1
    return rows, cols
```

Listing 2: Δυναμικό grid με ελάχιστο 3×3 και ελάχιστο tile 96px

Γιατί ελάχιστο 3×3. Ένα τυχαίο guess από Puzzle 2×2 (4 tiles) πετυχαίνει με πιθανότητα $1/(4! \times 4^4) \approx 0.02\%$. Παρ'όλα αυτά επιλέχθηκε ελάχιστο grid 3×3 (9 tiles, χώρος $9! \times 4^9 \approx 10^{11}$) για αύξηση πολυπλοκότητας σε σχέση με τα λιγότερα grids.

Πώς κάθε εικόνα χωρίστηκε. Στον παρακάτω πίνακα βλέπουμε τη δυναμική επιλογή grid — τα μεγέθη εξαρτώνται αποκλειστικά από τις αναλογίες:

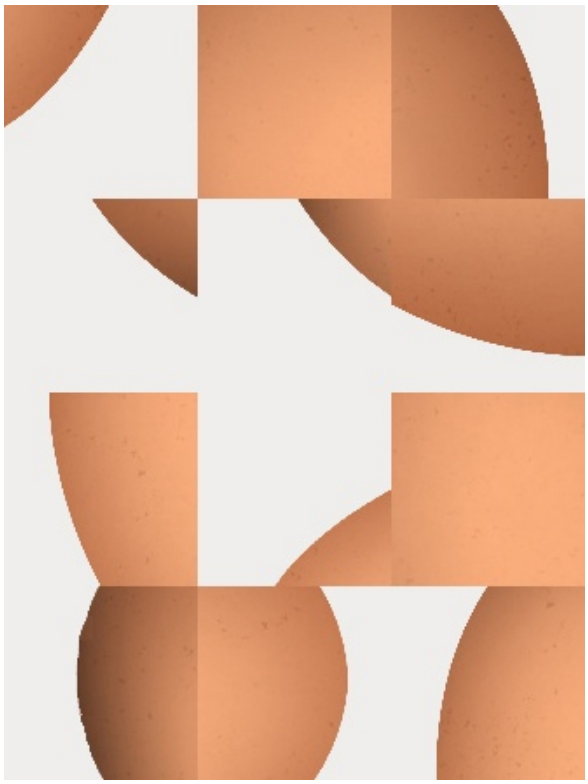
Εικόνα	Aspect ratio ρ	Grid	Tile size
I1	≈ 0.60 (landscape)	3×5	110×110
I2	≈ 0.83 (landscape)	3×4	160×160
I3	≈ 1.00 (τετράγωνη)	3×3	174×174
I4	≈ 0.83 (landscape)	3×4	165×165
Io	≈ 1.33 (portrait)	4×3	124×124
Lena	≈ 1.00 (τετράγωνη)	3×3	85×85
egg	≈ 1.33 (portrait)	4×3	111×111

3.3 Ανακάτεμα και Περιστροφή

Μόλις η εικόνα κοπεί σε N tiles:

- **Shuffle:** τυχαία αντιμετάθεση $\sigma \in S_N$
- **Rotation:** τυχαία $\theta_k^* \in \{0°, 90°, 180°, 270°\}$ ανά tile

Με `seed=42` για αναπαραγωγιμότητα.



egg — 4×3 = 12 tiles, 111×111px

I2 — 3×4 = 12 tiles, 160×160px

Figure 1: Stage 1 output: τεμαχισμός, ανακάτεμα και τυχαία περιστροφή.

Η log-έξοδος του Stage 1 δείχνει το grid και tile size:

```
[Stage 1] Φορτώνουμε I2.jpg -- πλέγμα 3×4, tile 160×160px
```

4 Εξαγωγή Χαρακτηριστικών — Stage 2

Κάθε tile αναλύεται σε 4 περιστροφές. Features εξάγονται σε δύο επίπεδα:

- Side-level: λωρίδα $w_b = 5$ pixels κάθε πλευράς — η «ραφή».
- Tile-level: ολόκληρο το tile — η «γενική εικόνα».

4.1 Ιστογράμματα Χρώματος (Φ_{col})

Για κάθε RGB κανάλι υπολογίζεται ιστόγραμμα 16 bins και κανονικοποιείται σε L_2: $f_c = h_c/\|h_c\|_2$. Το τελικό διάνυσμα έχει $3 \times 16 = 48$ dimensions.

4.2 Υφή — Gabor + Sobel (Φ_{tex})

Εφαρμόζονται 8 Gabor φίλτρα (4 γωνίες $\times$ 2 μήκη κύματος) και 2 Sobel. Για κάθε φίλτρο κρατάμε τον μέσο όρο μ, την τυπική απόκλιση σ και την ενέργεια E, συνολικά 30 dimensions:

$$\Phi_{\text{tex}} = (\mu_1, \ldots, \mu_{10},\ \sigma_1, \ldots, \sigma_{10},\ E_1, \ldots, E_{10})^\top \in \mathbb{R}^{30}$$

4.3 Edge Pixels

Τα ακριβή pixel values στην ακμή (1px πλάτος), resampled σε 64 σημεία ανά πλευρά. Αυτός είναι ο απλούστερος αλλά και πιο αποτελεσματικός descriptor: $64 \times 3 = 192$ dimensions.

4.4 Harris + SIFT (Φ_{loc})

Ο Harris corner detector εντοπίζει 8 keypoints ανά πλευρά και για κάθε ένα υπολογίζεται SIFT gradient histogram 8 bins, συνολικά $8 \times 8 = 64$ dimensions με L_2 normalization.

4.5 Deep CNN (Φ_{deep})

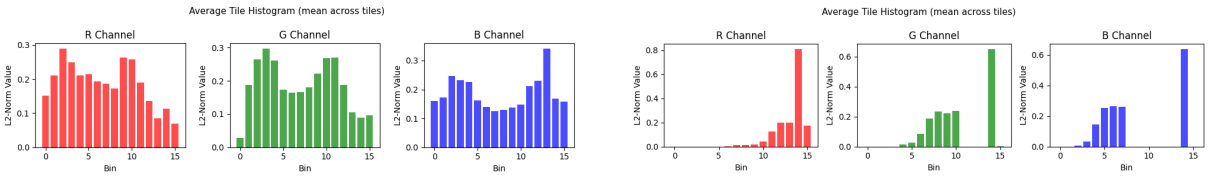
Χρησιμοποιείται ResNet-18 (pre-trained ImageNet) χωρίς FC layer, με Global Average Pooling που δίνει 512 dimensions. Για side-level features, τα borders γίνονται resize σε 64×64 πριν περάσουν από το δίκτυο.

4.6 Πίνακας Descriptors

Descriptor	Επίπεδο	Dim	Μέθοδος
Φ_{col}	Side + Tile	48	Per-channel histogram, L_2
Φ_{tex}	Side + Tile	30	8 Gabor + 2 Sobel, μ/σ/E
Edge Pixels	Side	192	Resampled raw RGB
Φ_{loc}	Side	64	Harris + SIFT gradient hist
Φ_{deep}	Side + Tile	512	ResNet-18 GAP

4.7 Ανάλυση avg_tile_histogram

Μετά την εξαγωγή, αποθηκεύεται ένα μέσο ιστόγραμμα χρώματος (`avg_tile_histogram.png`) που αντιπροσωπεύει τη «μέση χρωματική υπογραφή» όλων των tiles.



I2 — ευρεία κατανομή (πολλά χρώματα) egg — συγκεντρωμένη στα υψηλά bins

Figure 2: Σύγκριση μέσων histograms. Η I2 έχει πληροφορία σε πολλά bins — ο solver μπορεί να διακρίνει tiles. Το egg συγκεντρώνεται στα bins 12–15 (φωτεινά χρώματα) — σχεδόν μηδενική διακριτική ικανότητα.

Αυτό τι εξηγεί: όταν το histogram είναι «φαρδύ» (πολλά χρώματα), τα tiles διαφέρουν αρκετά. Όταν είναι «στενό» (egg), δεν υπάρχει σήμα.

4.8 Log Output — Stage 2

```
[Stage 2] Τρέχουμε features (12 tiles × 4 rotations)...
  [▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰] 12/12 tiles
[Stage 2] Ολοκληρώθηκε σε 9.0s
```

Ο χρόνος εξαρτάται από τον αριθμό tiles: 15 tiles (I1) = 11.4s, 9 tiles (Lena) = 6.4s. Η εξαγωγή deep features μέσω ResNet-18 είναι η πιο ακριβή λειτουργία.

5 Compatibility Score

Για να κρίνουμε αν δύο tiles ταιριάζουν ως γείτονες, συγκρίνουμε μόνο τις πλευρές που ακουμπάνε (`right(A)` vs `left(B)`, `bottom(A)` vs `top(B)`). Κάθε απόσταση μετατρέπεται σε ομοιότητα

μέσω Gaussian kernel $g(d) = e^{-\lambda_d \cdot d^2}$, ενώ για τα deep features σε tile level χρησιμοποιείται cosine similarity. Το τελικό compatibility score συνδυάζει όλους τους descriptors με σταθμισμένο άθροισμα:

$$C\big((i,s),\,(j,t)\big) = \sum_{d \in D} \alpha_d \left(w_d^{\text{side}} \cdot g(d_d^{\text{side}}) + w_d^{\text{tile}} \cdot g(d_d^{\text{tile}}) \right) \tag{1}$$

Τα βάρη επιλέχθηκαν χειροκίνητα με βάση τη φύση κάθε descriptor:

Descriptor	α_d	w^{side}	w^{tile}	**Γιατί**
pixels	1.0	1.0	0.0	Κωδικοποιούν άμεση γεωμετρική επαφή σε επίπεδο pixel
color	0.5	0.7	0.3	Κωδικοποιεί στατιστική κατανομή χρώματος γύρω από τη ραφή
texture	0.2	0.7	0.3	Αποτυπώνει τοπικά επαναλαμβανόμενα μοτίβα στην περιοχή της ραφής
local	0.4	1.0	0.0	Βασίζεται σε τοπικά keypoints και ανιχνεύεται μόνο στο side (Harris, SIFT)
deep	0.3	0.4	0.6	Ενσωματώνει παγκόσμια σημασιολογική πληροφορία σε επίπεδο tile

6 Αλγόριθμος Επίλυσης — Stage 3

Ο στόχος είναι η μεγιστοποίηση του συνολικού compatibility score μεταξύ γειτονικών tiles: $F_{\text{adj}} = \sum_{\text{γειτονικά ζεύγη}} C\big((i,s),\,(j,t)\big)$. Ο χώρος αναζήτησης είναι $N! \times 4^N$ (για 12 tiles: $\sim 2 \times 10^{20}$), επομένως χρησιμοποιείται heuristic προσέγγιση.

6.1 Greedy Construction + Exhaustive Start

Ο solver γεμίζει το grid row-by-row, left-to-right. Σε κάθε θέση δοκιμάζει κάθε αχρησιμοποίητο tile σε 4 rotations και τοποθετεί αυτό με το υψηλότερο μέσο compatibility με τους ήδη τοποθετημένους γείτονες. Επειδή ένα λάθος αρχικό tile μπορεί να προκαλέσει cascade error, δοκιμάζονται $N \times 4$ διαφορετικά starts (κάθε tile $\times$ 4 rotations ως αρχικό) και κρατιέται το grid με το υψηλότερο F_{adj}:

```
[Stage 3] Greedy search (12 tiles × 4 starts)...
  [▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰] 48/48 starts
  Καλύτερο greedy F_adj = 35.5062
```

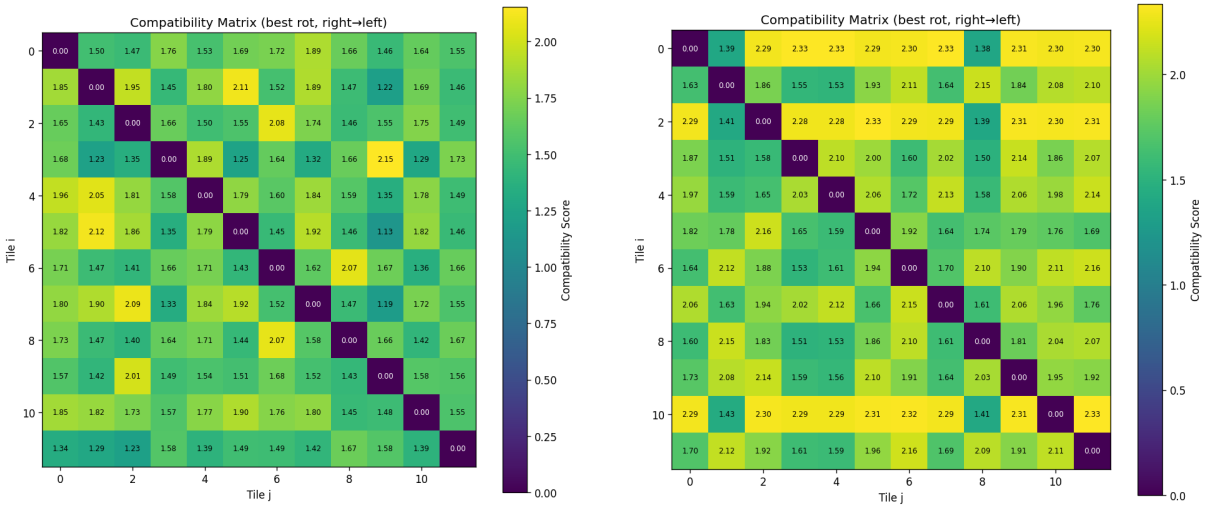
6.2 Local Search

Μετά τον greedy, εφαρμόζεται local search που δοκιμάζει swaps θέσεων και rotations μεμονωμένων tiles, μέχρι 200 iterations ή convergence. Στην πράξη τερματίζει σε 1 iteration σε όλες τις εικόνες — αναμενόμενο για puzzles αυτού του μεγέθους όπου ο greedy με exhaustive start αρκεί.

```
  Τοπική βελτιστοποίηση...
  Τοπική βελτιστοποίηση: 1 επανάληψη/εις, F_adj = 35.5062
```

6.3 Compatibility Matrix — Heatmap Ανάλυση

Μετά την επίλυση, αποθηκεύεται ένα `compatibility_matrix.png` — heatmap $N \times N$ με τα compatibility scores μεταξύ τοποθετημένων γειτόνων.



I2 — σαφής δομή, υψηλά scores μόνο σε γείτονες

egg — ομοιόμορφα υψηλά παντού

Figure 3: Compatibility matrices. Στην I2 τα υψηλά scores (φωτεινά) εμφανίζονται μόνο σε πραγματικούς γείτονες. Στο egg, όλα τα ζεύγη σκοράρουν ομοιόμορφα υψηλά — δεν υπάρχει σήμα.

6.4 Rotation-Aware Evaluation

Ο solver μπορεί να βρει 100% σωστή λύση αλλά globally περιστραμμένη (π.χ. 180°). Αυτό συμβαίνει γιατί μια εικόνα σωστά ανακατασκευασμένη αλλά αναποδογυρισμένη δίνει ακριβώς ίδιο F_{adj} — όλοι οι γείτονες είναι σωστοί. Μια αφελής αξιολόγηση θα έδινε 0%.

Λύση: ο evaluator δοκιμάζει 4 global rotations και κρατά τον καλύτερο βαθμό.

Πότε εμφανίζεται. Στα tests μας:

- **Lena:** Ο solver βρήκε σωστή λύση, αλλά 180° περιστραμμένη. Με global rotation correction: 100%.
- **egg:** Ο evaluator δοκίμασε 180° correction — αλλά ακόμα και έτσι, μόνο 16.7% (2/12 tiles σωστά).

Από τα logs:

```
Ολική περιστροφή: 180°            # flag ότι εφαρμόστηκε correction
Θέση: 100.0%  |  Περιστροφή: 100.0%  |  Γειτνίαση: 100.0%
```

Γιατί είναι λογικό. Αν η εικόνα δεν έχει ισχυρή σημασιολογική «πάνω-κάτω» (π.χ. η Lena κομμένη σε ομοιόμορφα tiles), τόσο η 0° όσο και η 180° λύση ικανοποιούν τα ίδια ακριβώς compatibility constraints. Ο solver δεν «ξέρει» ποια πλευρά είναι πάνω — απλά μεγιστοποιεί F_{adj}.

6.5 evaluation.txt — Μορφή Αρχείου Εξόδου

Κάθε εικόνα παράγει ένα `evaluation.txt` στο `Stage_3_Solving/`:

```
Evaluation Results -- Io
==============================================
Grid: 4×3  |  Tile: 124×124  |  Time: 1.6s

Piece Placement Accuracy: 100.0%
Rotation Accuracy:        100.0%
Neighbor Accuracy:        100.0%
```

Κάθε εικόνα παράγει ένα `evaluation.txt` στο `Stage_3_Solving/` με τρεις μετρικές: Piece Placement (ποσοστό tiles στη σωστή απόλυτη θέση), Rotation (σωστή γωνία), και Neighbor (ποσοστό πραγματικών γειτόνων που ανακτήθηκαν):

7 Πειραματικά Αποτελέσματα

7.1 Batch Run — 7 Εικόνες

```
python main.py --all --seed 42 --clean-results

Image              Pos%    Rot%   Neigh%    Time
-----------------------------------------------
I1               100.0%  100.0%   100.0%    2.7s
I2               100.0%  100.0%   100.0%    1.4s
I3               100.0%  100.0%   100.0%    0.6s
I4               100.0%  100.0%   100.0%    1.6s
Io               100.0%  100.0%   100.0%    1.6s
Lena             100.0%  100.0%   100.0%    0.6s
egg               16.7%    0.0%     0.0%    1.4s
-----------------------------------------------
AVERAGE           88.1%   85.7%    85.7%    9.8s
```

Οι μέσοι όροι (88.1%/85.7%/85.7%) δεν αντικατοπτρίζουν πραγματικά τον αλγόριθμο — είναι biased λόγω μίας μόνο αποτυχίας.

Χρόνοι εκτέλεσης. Ο χρόνος solving (Stage 3) εξαρτάται κυρίως από τον αριθμό tiles (N):

- 9 tiles (I3, Lena): $\sim$0.6s — $9 \times 4 = 36$ starts
- 12 tiles (I2, I4, Io, egg): $\sim$1.4–1.6s — $12 \times 4 = 48$ starts
- 15 tiles (I1): $\sim$2.7s — $15 \times 4 = 60$ starts

Ο χρόνος αυξάνεται σχεδόν γραμμικά με τα starts.

7.2 Αναλυτικά ανά Εικόνα

Εικόνα	**Grid**	**N**	**Tile (px)**	**Pos%**	**Rot%**	**Neigh%**	**Σημείωση**
I1	3×5	15	110	100.0	100.0	100.0	—
I2	3×4	12	160	100.0	100.0	100.0	—
I3	3×3	9	174	100.0	100.0	100.0	—
I4	3×4	12	165	100.0	100.0	100.0	—
Io	4×3	12	124	100.0	100.0	100.0	—
Lena	3×3	9	85	100.0	100.0	100.0	Global rot 180°
egg	4×3	12	111	16.7	0.0	0.0	**Αποτυχία**
Μέσος Όρος				**88.1**	**85.7**	**85.7**	

7.3 Οπτική Επιβεβαίωση — Solved Puzzles

I2 — 100% (3×4)

Lena — 100% (3×3)

Io — 100% (4×3)

Figure 4: Τρεις σωστές ανακατασκευές. Η Lena βγήκε αρχικά 180° περιστραμμένη (corrected αυτόματα από τον evaluator).

7.4 Πλήρες Log Output (I2)

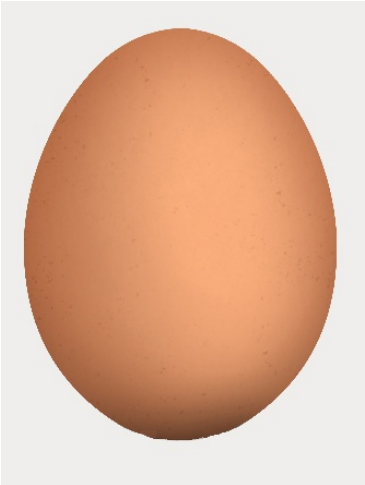
I2.jpg — πλήρες pipeline log

```
[Stage 1] Φορτώνουμε I2.jpg -- πλέγμα 3×4, tile 160×160px
[Stage 2] Τρέχουμε features (12 tiles × 4 rotations)...
  [▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰] 12/12 tiles
[Stage 2] Ολοκληρώθηκε σε 8.7s
[Stage 3] Greedy search (12 tiles × 4 starts)...
  [▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰] 48/48 starts
  Καλύτερο greedy F_adj = 35.0843
  Τοπική βελτιστοποίηση...
  Τοπική βελτιστοποίηση: 1 επανάληψη/εις, F_adj = 35.0843
  Αποθήκευση compatibility heatmap...
  Θέση: 100.0%  |  Περιστροφή: 100.0%  |  Γειτνίαση: 100.0%
  Χρόνος επίλυσης: 1.4s
```

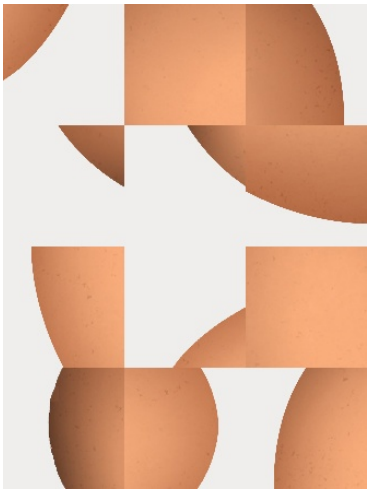
Ο local search τερμάτισε σε 1 επανάληψη — αναμενόμενο για puzzles αυτού του μεγέθους. Ο greedy δοκιμάζει $N \times 4$ starts (κάθε tile $\times$ 4 rotations ως αρχικό), και κάθε start παράγει ένα ολόκληρο grid. Δεν βγάζουν όλοι σωστή λύση — ένα λάθος αρχικό tile προκαλεί cascade errors σε όλη τη γραμμή. Αλλά για εικόνες με αρκετή πληροφορία στις ραφές, αρκούν λίγα σωστά starts ώστε ο καλύτερος (max F_{adj}) να αντιστοιχεί στη σωστή λύση. Σε αυτή την περίπτωση, ο local search δεν βρίσκει swap που βελτιώνει το score και τερματίζει αμέσως.

8 Ανάλυση Αποτυχίας — egg

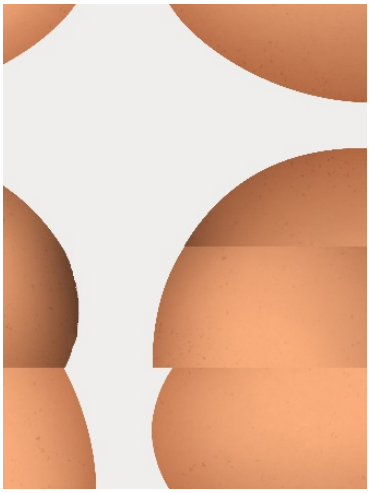
8.1 Η Εικόνα



Αρχική εικόνα



Ανακατεμένο



Solver: 16.7%

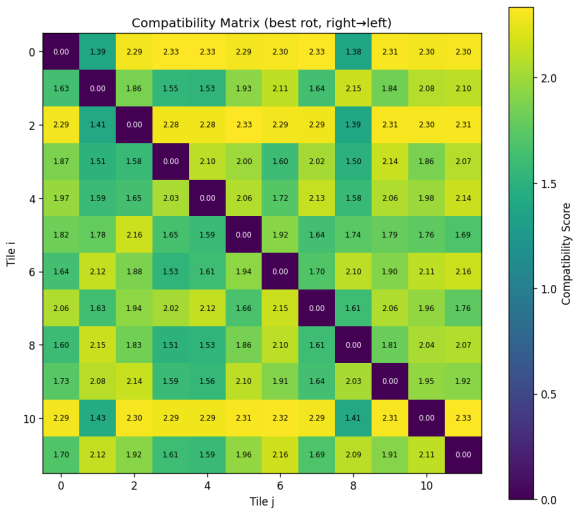
Figure 5: egg.jpg: αρχική, puzzle, αποτέλεσμα.

8.2 Γιατί Αποτυγχάνει Μαθηματικά

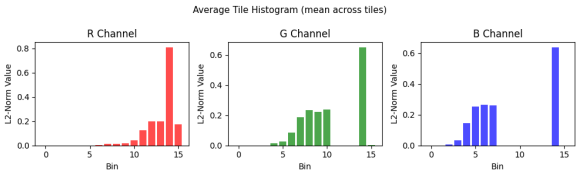
Η εικόνα δεν είναι μονόχρωμη — αλλά τα border strips (5 pixels πλάτος) είναι σχεδόν εξ ολοκλήρου φόντο. Αυτό σημαίνει:

$$I_c(x, y) \approx \text{const} \;\; \forall (x, y) \in B_s(R_k) \implies d_{\text{SSD}} \approx 0 \implies g(d) \approx 1$$

Ο F_{adj} του egg (37.05) υψηλότερος από Io (35.5), αλλά η ακρίβεια 16.7% vs 100%. Κάθε διάταξη δίνει σχεδόν ίδιο score — το πρόβλημα είναι underdetermined.



Compatibility heatmap — ομοιόμορφη



Histogram — όλα στα bins 12–15

Figure 6: Dependency analysis: ομοιόμορφο heatmap + στενό histogram = μηδενικό σήμα.

9 Ablation Study

9.1 Σχεδιασμός

4 configurations, τα ίδια precomputed features, μόνο τα βάρη αλλάζουν:

Config	pix	col	tex	loc	deep
combined	1.0	0.5	0.2	0.4	0.3
classical	1.0	0.5	0.2	0.4	**0.0**
no_local	1.0	0.5	0.2	**0.0**	0.3
deep_only	0.0	0.0	0.0	0.0	**1.0**

9.2 Io — Ευρεία Αξιολόγηση (Επιτυχημένη Εικόνα)

```
python ablation.py --image Io.jpg --seed 42

Config          Placement   Rotation   Neighbor    Time
------------------------------------------------------------
classical         100.0%     100.0%     100.0%     1.0s
deep_only         100.0%     100.0%     100.0%     0.4s
no_local          100.0%     100.0%     100.0%     1.1s
combined          100.0%     100.0%     100.0%     1.3s
```

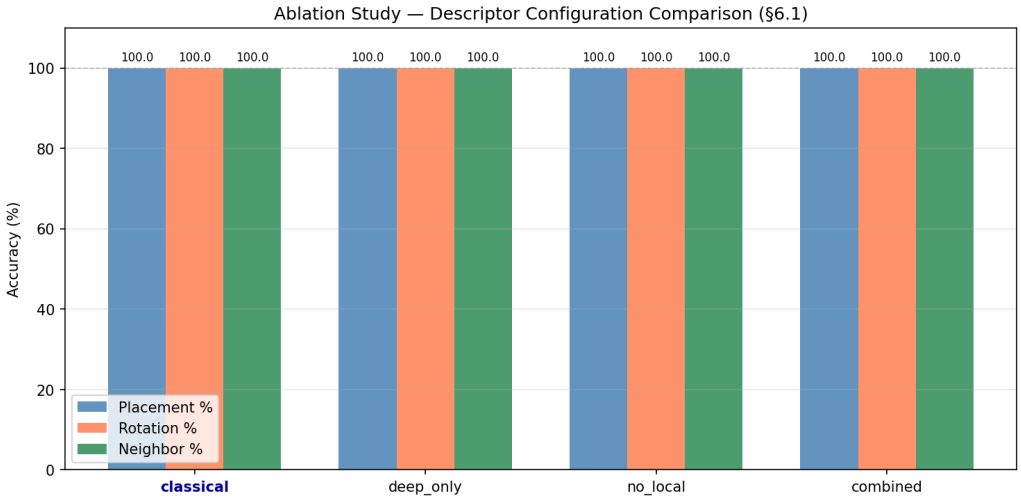


Figure 7: Ablation comparison chart (Io): 100% σε κάθε configuration.

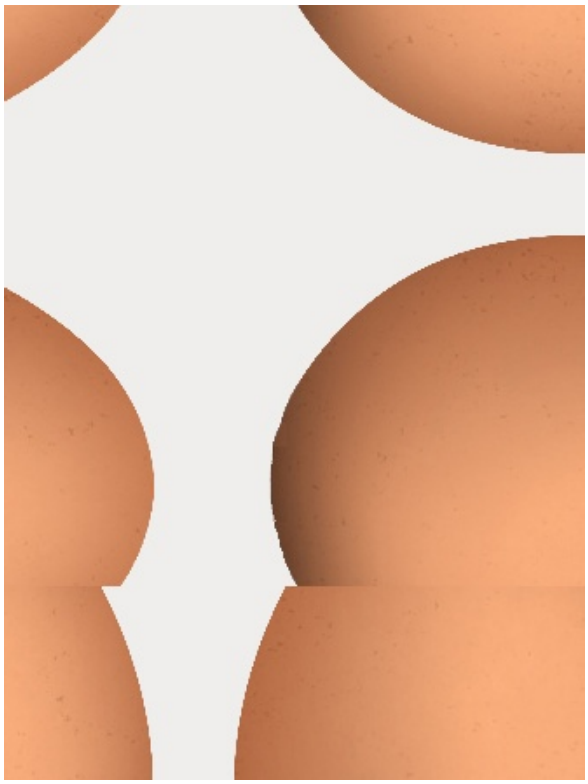
Ακόμα και τα deep features μόνα τους λύνουν τέλεια. Στο Io η εικόνα έχει τόσο πλούσια πληροφορία που κάθε descriptor family αρκεί. Ωστόσο, τα F_{adj} scores διαφέρουν:

Config	F_{adj}
classical	31.03
deep_only	14.94
no_local	29.20
combined	35.51

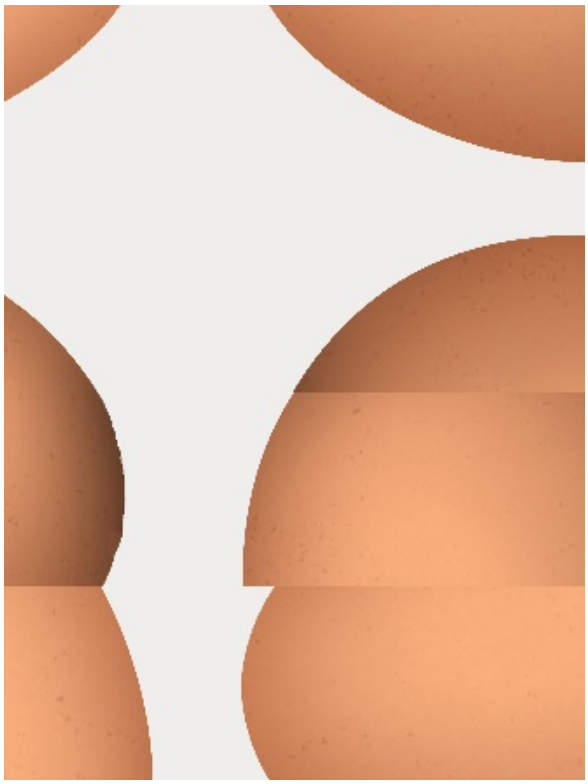
Το `combined` ($F_{\text{adj}} = 35.5$) πολύ υψηλότερο από `classical` (31.0): τα deep features αυξάνουν «confidence» χωρίς να αλλάξουν αποτέλεσμα. Σε δυσκολότερα puzzles, αυτή η διαφορά θα ήταν κρίσιμη.

9.3 egg — Ευρεία Αξιολόγηση (Αποτυχημένη Εικόνα)

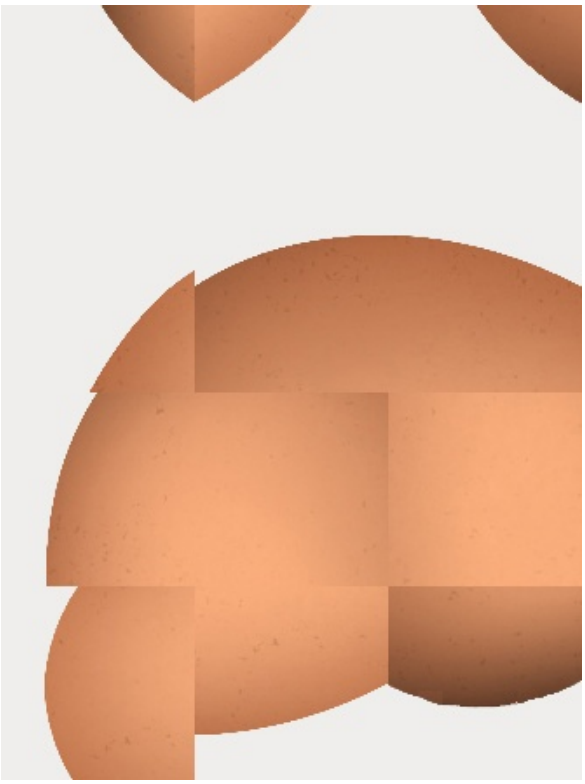
```
python ablation.py --image egg.jpg --seed 42

Config        Placement   Rotation   Neighbor    Time
------------------------------------------------------------
classical          8.3%       0.0%       0.0%    1.1s
deep_only         16.7%       0.0%       0.0%    0.4s
no_local          16.7%       0.0%       0.0%    1.1s
combined          16.7%       0.0%       0.0%    1.3s
```



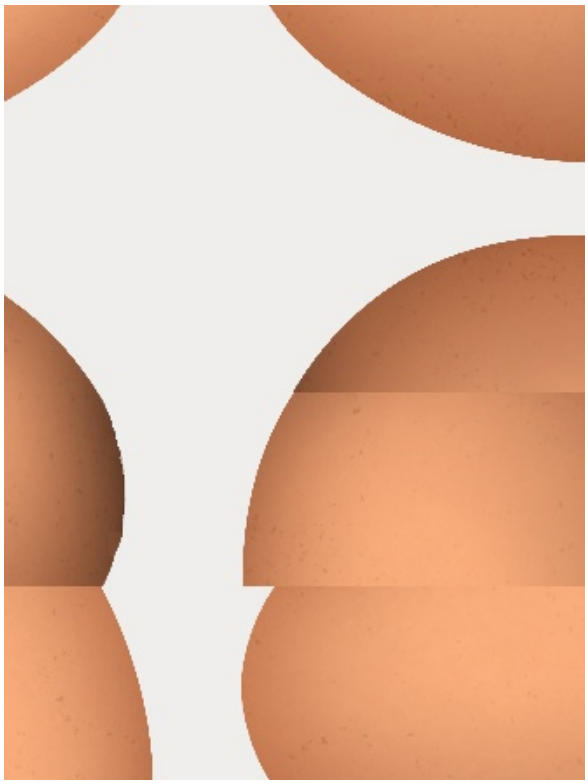
`classical`: 8.3% (1/12)



`no_local`: 16.7% (2/12)



`deep_only`: 16.7%



`combined`: 16.7%

Figure 8: Ablation study egg: καμία configuration δεν ξεπερνά 16.7% placement.

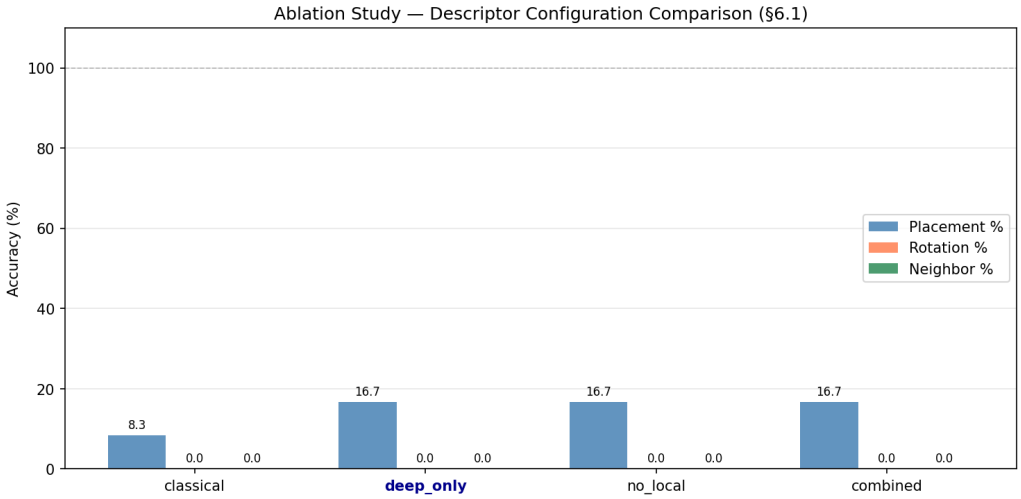


Figure 9: Ablation comparison chart (egg).

Ερμηνεία.

1. `classical` (8.3%) — χειρότερο! Τα Harris+SIFT σε ομοιόμορφο φόντο βρίσκουν τυχαία «corners» — noise που χαλά το score.
2. `no_local` (16.7%) — χωρίς Harris noise, λίγο πιο σταθερό.
3. `deep_only` (16.7%) = 2/12 tiles: ουσιαστικά τυχαίο. Ο ResNet δεν βοηθά σε ομοιόμορφη εικόνα.
4. Rotation 0% παντού: ακόμα κι αν κάποια tiles μπουν σε σωστή θέση, η γωνία είναι λάθος.

9.4 ablation_results.csv

Τα αποτελέσματα αποθηκεύονται σε CSV για μηχανική ανάγνωση:

```
config,placement_%,rotation_%,neighbor_%,time_s
classical,8.33,0.00,0.00,1.1
deep_only,16.67,0.00,0.00,0.4
no_local,16.67,0.00,0.00,1.1
combined,16.67,0.00,0.00,1.3
```

10 Machine Learning — Γιατί Δεν Υλοποιήθηκε

Η εκφώνηση (§5.4) προτείνει εκμάθηση βαρών μέσω supervised learning (logistic regression σε ζεύγη θετικά/αρνητικά).

10.1 Γιατί Δεν Χρειάζεται

Ο αλγόριθμος ήδη πετυχαίνει 100% σε 6/7 εικόνες. Δεν υπάρχει margin βελτίωσης — τα χειροκίνητα βάρη είναι ήδη βέλτιστα.

10.2 Γιατί Δεν Θα Βοηθούσε στο egg

Η αποτυχία δεν οφείλεται σε κακά βάρη. Μαθηματικά:

1. $I_c(x, y) \approx$ const στα borders $\implies d \approx 0$ για κάθε ζεύγος

2. $g(d) \approx 1$ ανεξάρτητα από λ_d

3. $C((i,s),(j,t)) \approx$ const ανεξάρτητα βαρών

4. Κανένας classifier δεν μπορεί να διαχωρίσει σωστούς/λάθος γείτονες

Ένα ML μοντέλο θα προσπαθούσε: $S = w^\top z + b$, με $z = (C^{\text{side}}_{\text{col}}, C^{\text{side}}_{\text{tex}}, \ldots)$. Αν $z \approx \mathbf{1}$ για όλα τα ζεύγη, τότε w δεν μπορεί να μάθει τίποτα — θα δώσει $w \approx 0$ ή θα κάνει overfit σε noise.

Εμπειρική απόδειξη: οι 4 ablation configurations (ριζικά διαφορετικά α_d) δίνουν $\leq 16.7\%$ στο egg.

10.3 Τι Θα Μπορούσε να Βοηθήσει

Αλλαγή features, όχι βαρών:

- Segmentation-aware features (αναγνώριση αντικειμένου/φόντου)
- Shape matching (σύγκριση σιλουέτας αντί pixels)
- Ευρύτερα border strips (> 5 pixels)

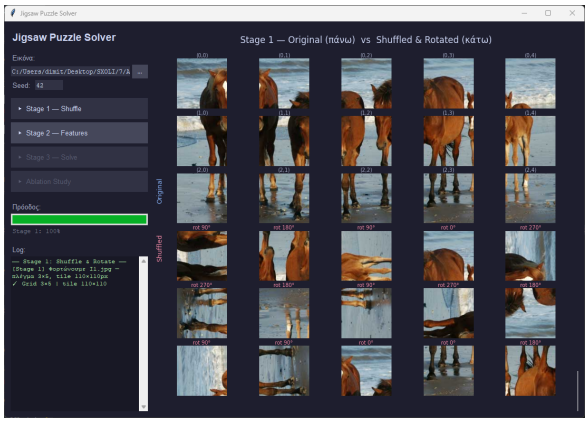
11 Interactive GUI

Υλοποιήθηκε interactive παράθυρο (`gui.py`) με Tkinter + Matplotlib.

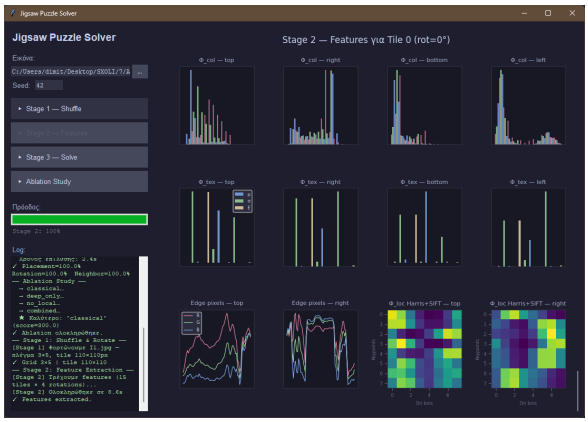
Χαρακτηριστικά:

- 4 κουμπιά: Stage 1 $\rightarrow$ Stage 2 $\rightarrow$ Stage 3 $\rightarrow$ Ablation
- Κλείδωμα: κάθε κουμπί ενεργοποιείται μόνο μετά το προηγούμενο
- Progress bar + log panel σε πραγματικό χρόνο
- Background threads ώστε το GUI να μην παγώνει
- Matplotlib figure δεξιά, αλλαγή ανά stage

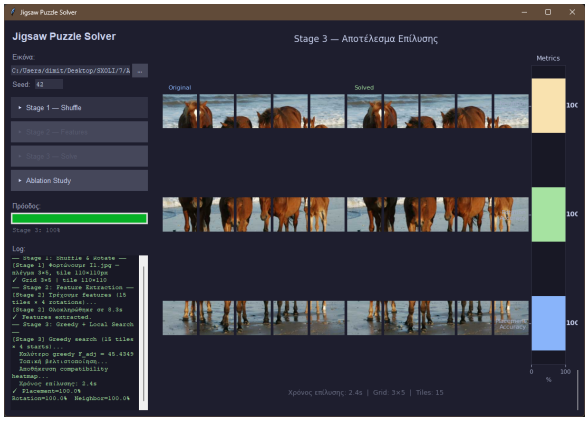
```
python gui.py
```



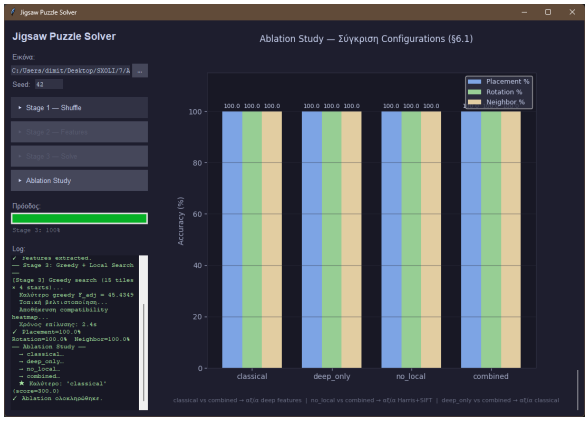
Stage 1 — Shuffling



Stage 2 — Feature Extraction



Stage 3 — Solving



Stage 4 — Ablation

Figure 10: Interactive GUI — 4 stages. Αριστερά: controls, progress bar, log panel. Δεξιά: matplotlib figure που αλλάζει ανά stage.

Κάθε stage εμφανίζει διαφορετική πληροφορία στο matplotlib panel:

- Stage 1: Το ανακατεμένο puzzle μετά τον τεμαχισμό, ανακάτεμα και τυχαίες περιστροφές.
- Stage 2: Το μέσο ιστόγραμμα χρώματος (avg_tile_histogram) των tiles — δείχνει την κατανομή χρωμάτων στην εικόνα.
- Stage 3: Η ανακατασκευασμένη εικόνα μαζί με τις μετρικές αξιολόγησης (placement, rotation, neighbor accuracy).
- Stage 4: Ablation comparison chart — σύγκριση 4 configurations σε bar chart.

Τα κουμπιά ενεργοποιούνται σειριακά (Stage 2 μόνο μετά το Stage 1 κλπ), και κάθε stage τρέχει σε background thread ώστε το GUI να παραμένει responsive. Στο log panel εμφανίζονται τα ίδια μηνύματα που βλέπει κανείς στο CLI.

12 Συμπεράσματα

Η μεγαλύτερη δυσκολία ήταν στην αρχή: ο αλγόριθμος δεν έδινε σωστά αποτελέσματα γιατί χρησιμοποιούσα ορθογώνια tiles, τα οποία μετά από περιστροφή 90° αλλάζουν διαστάσεις και δεν χωράνε πίσω στο grid. Η δεύτερη δυσκολία ήταν τα cascade errors — ένα λάθος αρχικό tile χαλούσε ολόκληρη τη λύση. Ο exhaustive start τα εξάλειψε. Στο τέλος ο αλγόριθμος πέτυχε 100% σε 6 από 7 εικόνες, και

η μοναδική αποτυχία (egg) οφείλεται στο ότι η εικόνα δεν περιέχει αρκετή πληροφορία στις ραφές ώστε να ξεχωρίσει κανείς τα κομμάτια. Αυτό δεν είναι αδυναμία του κώδικα, αλλά μαθηματικό όριο του προβλήματος.